

Project Review 1 Report

Project Title: Adaptive Routing in Energy-Harvesting Wireless Sensor Networks

Student Name: Santhosh

Course: Data Structures & Algorithms (2nd Year B.Tech)

Date: August 2026

1. Problem Statement & Objectives

In standard Wireless Sensor Networks (WSNs), sensor nodes are powered by non-rechargeable batteries. Once a node's energy drops below a threshold, routing algorithms treat it as near-depleted and route traffic away from it.

When nodes are equipped with solar or ambient RF harvesters, battery levels fluctuate. A node that is low on energy right now may recharge in the next few time intervals. Standard shortest-path algorithms (like Dijkstra) or static bottleneck dynamic programming (DP) do not look ahead to future energy intake, leading to sub-optimal route choices.

Main Objectives:

- Build a Python simulation to model sensor node communication and energy harvesting.
- Apply DSA techniques (Graphs, Heaps, Dynamic Programming, and Disjoint-Set Union) to optimize routing in harvesting conditions.
- Implement and test a Time-Augmented DP algorithm that incorporates future energy recharge when selecting relay paths.
- Add live detour recovery using Union-Find to handle unexpected intermediate node battery failures.
- Compare baseline, unaware, and harvesting-aware configurations.

2. DSA Modules Implemented

- **Graph Representation** (`network.py`): Adjacency list storing sensor nodes and distance-weighted edges.
- **First-Order Radio Model** (`energy_model.py`): Calculates transmit and receive energy based on bit count and distance (d^2 free space / d^4 multipath).
- **Harvesting Profiles** (`harvesting_model.py`): Models diurnal solar cycles and stochastic Poisson energy arrivals.
- **LEACH Clustering with Min-Heap** (`clustering.py`): Cluster head election weighted by projected energy over the round.
- **Shortest Path & Rerouting** (`routing.py`):

- Dijkstra and A* algorithms for minimum-energy pathfinding.
- Disjoint-Set Union (Union-Find) with path compression and rank optimization for local detour splicing.
- **Time-Augmented DP** (`dp_lifetime.py`): 3D dynamic programming table `dp[node][hops][time]` to maximize bottleneck path energy considering future energy recharge.
- **Simulator & Visualizer** (`simulator.py`, `visualize.py`): Round-based simulation driver and Matplotlib plotting routines.

3. Review 1 Status & Test Results

3.1 Unit Testing

All 31 unit tests across the 9 modules pass using pytest:

- Radio energy calculations and threshold checks
- Graph construction and energy deduction
- Cluster head rotation and harvest weighting
- Dijkstra and A* path parity
- Union-Find connectivity and detour recovery
- Time-Augmented DP table fill and path backtracking

3.2 Experimental Run (50 Nodes, 350 Rounds)

Configuration	First Node Death (FND)	Half Nodes Dead (HND)	Alive Nodes (Round 350)	Total Residual Energy
:---	:---	:---	:---	:---
1. Baseline (No Harvesting)	Round 78	Round 109	0 / 50	0.0000 J
2. Solar (Unaware LEACH + Dijkstra)	Round 135	Round 208	1 / 50	0.0197 J
3. Solar (Adaptive Time-DP)	Round 167	Round 208	0 / 50	0.0000 J
4. Stochastic Poisson (Unaware LEACH)	Round 300	N/A	42 / 50	0.2224 J
5. Stochastic Poisson (Adaptive Time-DP)	Round 301	N/A	44 / 50	0.2296 J

:---

4. Key Takeaways & Plan for Review 2

1. Observations:

- Solar harvesting extends initial node survival by over 70% compared to no harvesting.

- Time-Augmented DP achieves better energy preservation under stochastic energy arrivals by actively utilizing nodes during recharge bursts.

2. Next Steps for Review 2:

- Test on irregular / non-uniform node deployments.
- Evaluate performance under variable data packet sizes and burst traffic.